

ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas

MANUAL TÉCNICO

Calculadora de Conversión de Bases Numéricas

Arquitectura MVC en Java con Swing

Asignatura: ICCD332 — Arquitectura de Computadores

Grupo: GR1SW — 2026-A

Repositorio:

github.com/jhosting23/ARQUITECTURA-DE-COMPUTADORES-ICCD332-GR1SW_2026-A

Quito, Ecuador — Mayo 2026

I. INTRODUCCIÓN

El presente Manual Técnico documenta la arquitectura interna, el diseño orientado a objetos y la estructura del código fuente del proyecto «Calculadora de Conversión de Bases Numéricas», desarrollado para la asignatura ICCD332 — Arquitectura de Computadores (GR1SW, 2026-A) de la Escuela Politécnica Nacional. El documento está dirigido a desarrolladores, docentes y cualquier profesional que deba comprender, mantener o extender el sistema.

La aplicación permite convertir números enteros entre las bases Binaria (2), Octal (8), Decimal (10) y Hexadecimal (16) mediante una interfaz gráfica de usuario (GUI) implementada con Java Swing. El sistema adopta el patrón arquitectónico Modelo-Vista-Controlador (MVC) y aplica los principios fundamentales de la Programación Orientada a Objetos (POO).

II. ALCANCE

Este manual abarca los siguientes aspectos técnicos del proyecto:

- Descripción de la arquitectura MVC y los paquetes del sistema.
- Diagrama de clases UML con relaciones entre componentes.
- Descripción detallada de cada clase, sus atributos y métodos.
- Explicación de los principios POO aplicados en el código fuente real.
- Flujo de datos entre capas: Vista → Controlador → Modelo.
- Manejo de excepciones y validaciones implementadas.
- Requisitos del entorno de compilación y ejecución.

III. REQUISITOS DEL SISTEMA

3.1 Requisitos de Software

Componente	Descripción
JDK 11 o superior	Compilación y ejecución de clases Java
IDE: IntelliJ IDEA / VS Code / Eclipse	Edición, compilación y depuración
Git 2.x	Clonación del repositorio y control de versiones
Sistema Operativo	Windows 10/11, Ubuntu 20.04+, macOS 12+

3.2 Requisitos de Hardware

Componente	Especificación
Procesador	1 GHz o superior (arquitectura x64)
Memoria RAM	512 MB mínimo — 2 GB recomendado para el IDE
Espacio en disco	100 MB libres (JDK + proyecto)
Pantalla	Resolución mínima 800 × 600 px

IV. ARQUITECTURA DEL SISTEMA

4.1 Patrón MVC

El sistema implementa el patrón Modelo-Vista-Controlador (MVC), descrito por Gamma et al. [1] como uno de los patrones de diseño arquitectónico de mayor impacto. La separación de responsabilidades reduce el acoplamiento entre componentes y facilita el mantenimiento y la prueba independiente de cada capa [2].

Capa MVC	Clase	Responsabilidad
Modelo	ConvertidorBase	Lógica de conversión entre bases. No tiene dependencias de la Vista ni del Controlador.
Vista	VistaCalculadora	Interfaz gráfica Swing. Presenta datos y captura eventos del usuario.
Controlador	ControlCalculadora	Intermediario: lee la Vista, invoca el Modelo y actualiza la Vista con el resultado.
Punto entrada	APP	Instancia los tres componentes y lanza la interfaz gráfica en el Event Dispatch Thread.

4.2 Estructura de Paquetes

La versión final del proyecto (archivos _1_.java) adopta la siguiente organización de paquetes Java:

Calculadora/		
├── backend/		
│ ├── ConvertidorBase.java	← Modelo (lógica pura)	
├── controlador/		
│ ├── ControlCalculadora.java	← Controlador (MVC)	
├── frontend/		
│ ├── VistaCalculadora.java	← Vista (GUI Swing)	
└── APP.java	← Punto de entrada (main)	

Nota: El repositorio también contiene versiones previas (ControlCalculadora.java, ConvertidorBase.java, VistaCalculadora.java, APP.java) con el paquete Calculadora.conversion.bases. Estas corresponden a una iteración anterior sin separación de capas en subdirectorios. La versión vigente es la que utiliza los subpaquetes backend, controlador y frontend.

V. DIAGRAMA DE CLASES UML

El diagrama siguiente emplea notación UML 2.5 [3]. Las flechas de asociación indican que ControlCalculadora mantiene referencias a ConvertidorBase (modelo) y a VistaCalculadora

(vista). La clase APP solo instancia los tres componentes y no forma parte de la jerarquía de herencia.

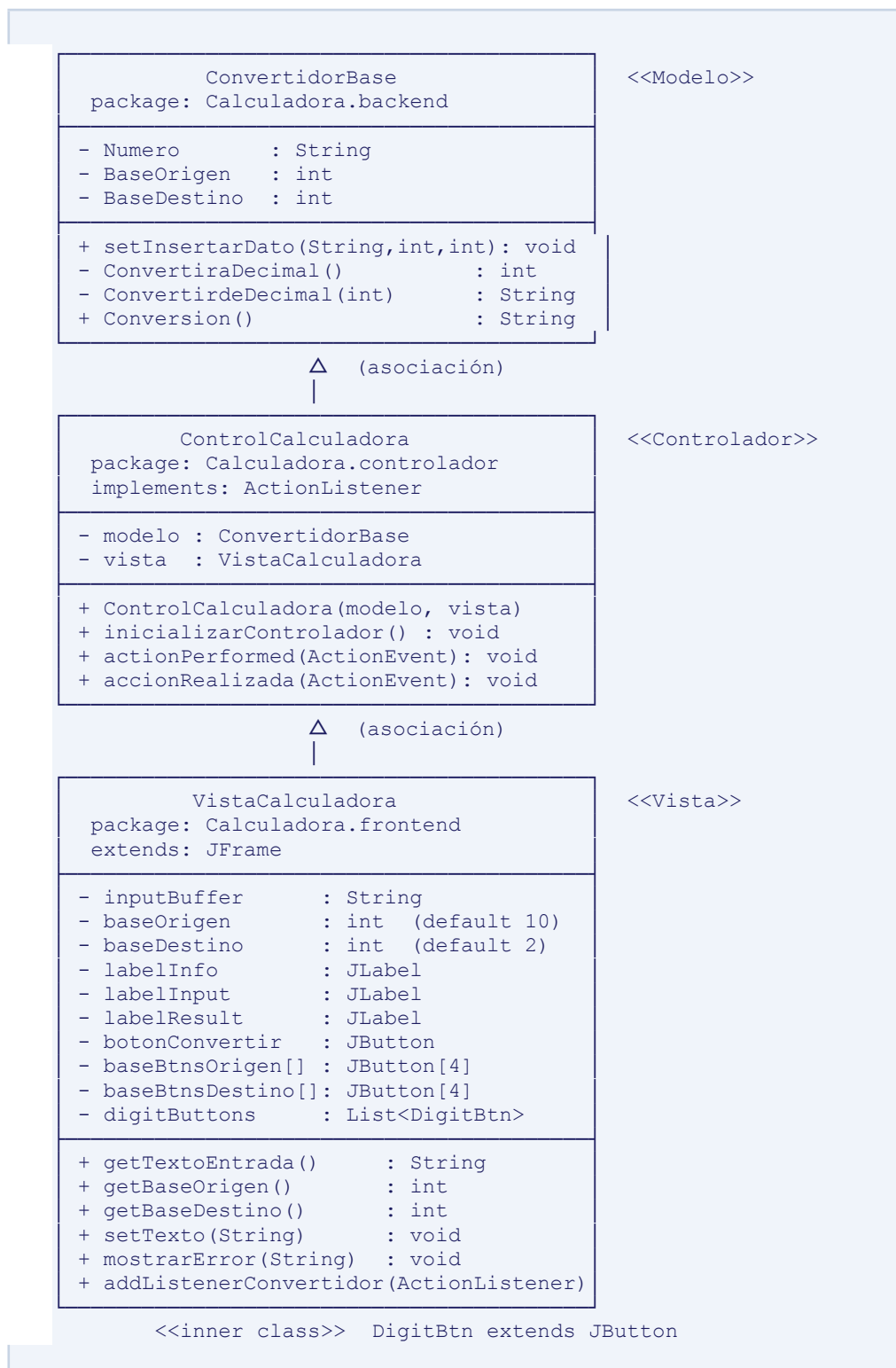


Figura 1. Diagrama de clases UML — Calculadora de Conversión de Bases (MVC).

VI. DESCRIPCIÓN DETALLADA DE CLASES

6.1 Clase ConvertidorBase (Modelo)

Clase POJO (Plain Old Java Object) que encapsula toda la lógica de negocio de la conversión numérica. No extiende ninguna clase de interfaz gráfica ni importa bibliotecas de presentación, lo que garantiza su independencia y facilita las pruebas unitarias [4].

Miembro	Tipo	Visib.	Descripción
Numero	String	Privado	Número a convertir, representado como cadena para admitir dígitos hexadecimales (A–F).
BaseOrigen	int	Privado	Base del sistema numérico de entrada (2, 8, 10 o 16).
BaseDestino	int	Privado	Base del sistema numérico de salida (2, 8, 10 o 16).
setInsertarDato()	void	Público	Setter que carga los tres parámetros de conversión en un solo llamado.
ConvertiraDecimal()	int	Privado	Utiliza Integer.parseInt(String, int) para convertir Numero a decimal según BaseOrigen.
ConvertirdeDecimal()	String	Privado	Si BaseDestino == 10 retorna String.valueOf(); de lo contrario usa Integer.toString(int, int).toUpperCase().
Conversion()	String	Público	Método de fachada que invoca la cadena de conversión y atrapa NumberFormatException, relanzándola como IllegalArgumentException con mensaje descriptivo.

6.2 Clase VistaCalculadora (Vista)

Extiende JFrame e implementa la interfaz gráfica completa. Utiliza GridBagLayout para el panel de botones y BorderLayout para el panel de display. Implementa el principio de responsabilidad única (SRP) [5]: solo gestiona la presentación y la captura de entrada, sin contener lógica de negocio.

6.2.1 Componentes visuales principales

Componente	Tipo Java	Descripción
labelInfo	JLabel	Muestra la dirección de conversión: «De: Decimal (10) → A: Binario (2)».
labelInput	JLabel	Display principal (40 pt negrita) que muestra el número ingresado o «0».

labelResult	JLabel	Display secundario (15 pt Consolas) que muestra el resultado con prefijo «→».
botonConvertir	JButton	Botón «= CONVERTIR» que ocupa el ancho completo (5 columnas GridBag).
baseBtnsOrigen[4]	JButton[]	Fila de 4 botones BIN/OCT/DEC/HEX para seleccionar la base de origen.
baseBtnsDestino[4]	JButton[]	Fila de 4 botones BIN/OCT/DEC/HEX para seleccionar la base de destino.
digitButtons	List<DigitBtn>	Lista de botones numéricos (0–9) y hexadecimales (A–F). Se deshabilitan automáticamente según la base de origen activa.

6.2.2 Clase interna DigitBtn

Clase privada anidada que extiende JButton y sobrescribe paintComponent(Graphics g) para renderizar botones con esquinas redondeadas, gradiente y efecto de hover/press. Almacena el valor entero del dígito (digitValue) para que refreshDigitStates() pueda compararlo con baseOrigen y habilitar o deshabilitar el botón según la base activa.

6.2.3 Métodos de la API pública

Método	Retorno	Descripción
getTextoEntrada()	String	Retorna inputBuffer: la cadena acumulada de dígitos ingresados.
getBaseOrigen()	int	Retorna el valor entero de la base de origen seleccionada.
getBaseDestino()	int	Retorna el valor entero de la base de destino seleccionada.
setTexto(String r)	void	Establece labelResult con el formato «→ r» en color cian TEXT_RESULT.
mostrarError(String msg)	void	Pone «Error» en labelResult (color rojo) y lanza un JOptionPane de error.
addListenerConvertidor(ActionListener)	void	Registra el ActionListener del Controlador en botonConvertir.

6.3 Clase ControlCalculadora (Controlador)

Implementa la interfaz ActionListener de Java AWT [6], lo que le permite suscribirse directamente al evento clic del botón «CONVERTIR». El constructor recibe ambas dependencias por inyección (Dependency Injection), siguiendo el principio de inversión de dependencias (DIP) [5].

Miembro	Tipo / Retorno	Descripción
modelo	ConvertidorBase	Referencia al Modelo. Se inyecta en el constructor.
vista	VistaCalculadora	Referencia a la Vista. Se inyecta en el constructor.
inicializarControlador()	void	Registra el propio Controlador como listener del botón CONVERTIR vía <code>vista.addListenerConvertidor(this)</code> .
actionPerformed(e)	void	Delegación obligatoria de <code>ActionListener</code> . Llama internamente a <code>accionRealizada(e)</code> .
accionRealizada(e)	void	Orquesta el flujo: lee Vista → valida entrada → invoca Modelo → actualiza Vista (resultado o error).

6.4 Clase APP (Punto de Entrada)

Contiene únicamente el método `main(String[] args)`. Instancia el Modelo, la Vista y el Controlador en ese orden —obligatorio porque `ControlCalculadora` requiere ambos como parámetros— y llama a `vista.setVisible(true)` para mostrar la GUI. En la versión refactorizada (`_1_`) los imports apuntan a los subpaquetes `backend`, `controlador` y `frontend`.

```
public static void main(String[] args) {
    ConvertidorBase modelo = new ConvertidorBase();
    VistaCalculadora vista = new VistaCalculadora();
    ControlCalculadora control = new ControlCalculadora(modelo,
    vista);
    vista.setVisible(true);
}
```

VII. PRINCIPIOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

7.1 Encapsulamiento

En `ConvertidorBase` los tres atributos (`Numero`, `BaseOrigen`, `BaseDestino`) son privados. El acceso externo se realiza exclusivamente a través del método público `setInsertarDato()` y del método `Conversion()`. Los métodos de conversión interna (`ConvertiraDecimal`, `ConvertirdeDecimal`) son también privados, ocultando los detalles de implementación al Controlador y a la Vista [7].

7.2 Abstracción

VistaCalculadora define una interfaz pública mínima (get/set/addListener) que es todo lo que el Controlador necesita conocer. El Controlador ignora por completo la implementación interna de la GUI: no sabe si los datos vienen de un JTextField, un JLabel o cualquier otro componente Swing. Esto reduce el acoplamiento y facilita el reemplazo de la Vista por otra implementación sin modificar el Controlador [2].

7.3 Herencia

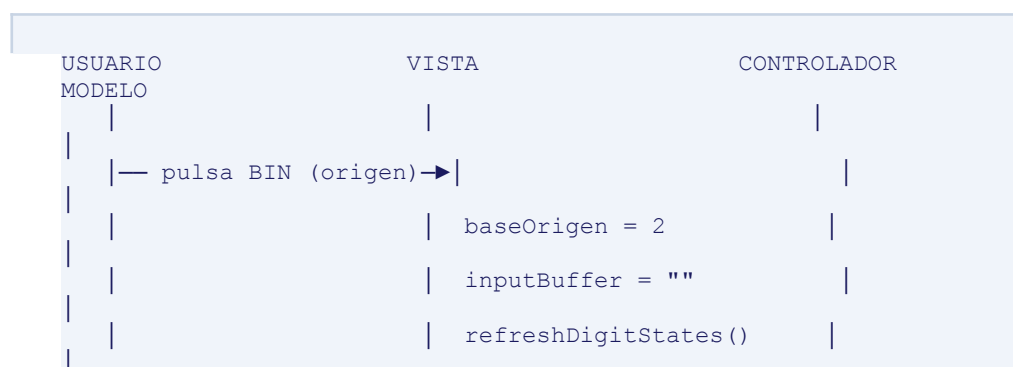
VistaCalculadora hereda de javax.swing.JFrame, aprovechando la infraestructura de ventana del framework. La clase interna DigitBtn hereda de JButton y sobrescribe paintComponent(Graphics g) para personalizar la apariencia con esquinas redondeadas y gradiente, extendiendo el comportamiento de la clase padre sin modificarla (Principio Abierto/Cerrado — OCP) [5].

7.4 Polimorfismo

ControlCalculadora implementa la interfaz ActionListener. Al registrarse como listener del botón, la JVM invoca actionPerformed(ActionEvent) de forma polimórfica sin conocer el tipo concreto del listener. Esto permite que en el futuro se puedan registrar múltiples listeners o reemplazar el Controlador por otra clase que implemente ActionListener sin alterar la Vista ni el Modelo [6].

VIII. FLUJO DE DATOS ENTRE CAPAS

El siguiente esquema describe el ciclo completo de una conversión exitosa desde la perspectiva del flujo de datos:



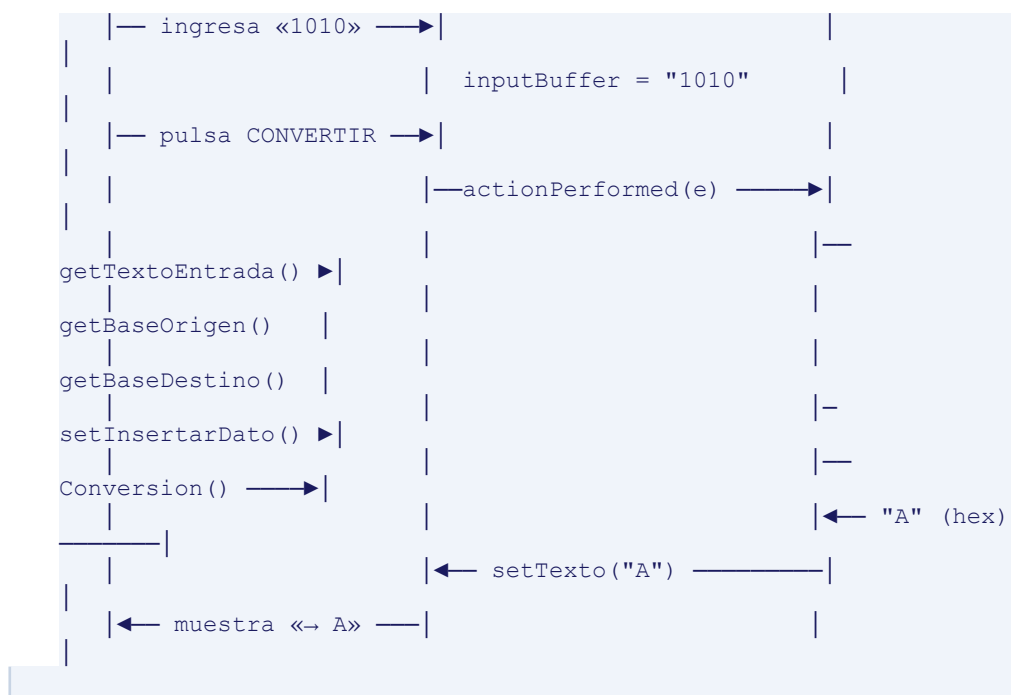


Figura 2. Diagrama de secuencia simplificado — conversión Binario→Hexadecimal de «1010».

IX. MANEJO DE EXCEPCIONES Y VALIDACIONES

9.1 Jerarquía de Excepciones

El sistema implementa un manejo estructurado de errores en dos niveles:

- Nivel Modelo: `ConvertidorBase.Conversion()` atrapa `NumberFormatException` (lanzada por `Integer.parseInt` cuando la cadena contiene dígitos inválidos para la base) y la reenvuelve como `IllegalArgumentException` con un mensaje descriptivo en español.
- Nivel Controlador: `ControlCalculadora.accionRealizada()` atrapa `IllegalArgumentException` y delega el mensaje a `vista.mostrarError()`, manteniendo el Modelo libre de lógica de presentación.

9.2 Validaciones en el Controlador

Caso de error	Condición detectada	Respuesta del sistema
Entrada vacía	<code>numero == null numero.trim().isEmpty()</code>	<code>vista.mostrarError("Debe ingresar un número")</code>
Dígito inválido para base	<code>Integer.parseInt</code> lanza <code>NumberFormatException</code>	Atrapado → <code>IllegalArgumentException</code> → <code>mostrarError</code>

Desbordamiento entero	Número mayor que Integer.MAX_VALUE	NumberFormatException → IllegalArgumentException → mostrarError
-----------------------	------------------------------------	---

9.3 Deshabilitación Preventiva de Botones

La Vista implementa `refreshDigitStates()`, que recorre la lista `digitButtons` y compara `digitValue < baseOrigen`. Si el dígito no pertenece a la base activa, el botón se deshabilita visualmente (`setEnabled(false)`), impidiendo la entrada de dígitos inválidos antes de que lleguen al Modelo. Esta es una validación de primera línea en la capa de presentación [8].

X. ESTÁNDARES DE CODIFICACIÓN

Convención	Estilo adoptado	Ejemplo en el código
Nombres de clases	PascalCase	ConvertidorBase, VistaCalculadora, ControlCalculadora
Nombres de métodos	camelCase	setInsertarDato(), getTextoEntrada(), accionRealizada()
Nombres de atributos	camelCase / PascalCase	inputBuffer, baseOrigen / Numero, BaseOrigen (versión original)
Constantes	UPPER_SNAKE_CASE	BLUE, BTN_EQ, BTN_BASE_SEL, BASE_SHORT[]
Paquetes	minúsculas con punto	Calculadora.backend, Calculadora.controlador, Calculadora.frontend
Codificación fuente	UTF-8	Soporta tildes y caracteres especiales del español

XI. REFERENCIAS

- [1] E. Gamma, R. Helm, R. Johnson y J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, EE. UU.: Addison-Wesley, 1994.
- [2] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, EE. UU.: Addison-Wesley, 2002.
- [3] Object Management Group (OMG), Unified Modeling Language Specification, versión 2.5.1. Needham, MA, EE. UU.: OMG, 2017.
- [4] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ, EE. UU.: Prentice Hall, 2008.

- [5] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River, NJ, EE. UU.: Prentice Hall, 2017.
- [6] Oracle Corporation, «java.awt.event.ActionListener», Java SE 17 API Documentation. [En línea]. Disponible: <https://docs.oracle.com/en/java/docs/>. [Accedido: 20 may. 2026].
- [7] G. Booch, R. Maksimchuk, M. Engle, B. Young, J. Conallen y K. Houston, Object-Oriented Analysis and Design with Applications, 3.^a ed. Upper Saddle River, NJ, EE. UU.: Addison-Wesley, 2007.
- [8] Oracle Corporation, «Creating a GUI with Swing», The Java Tutorials. [En línea]. Disponible: <https://docs.oracle.com/javase/tutorial/uiswing/>. [Accedido: 20 may. 2026].